

Relatório de Auditoria de Segurança

— Verzel Pass Redesign —

Data: 28/08/2026 | Auditoria de código-fonte estática

Escopo: API Fastify (TypeScript) + Frontend Next.js 16

Stack: Fastify 5 · Prisma ORM · PostgreSQL · JWT (cookie httpOnly) · Next.js 16 · TypeScript · Docker Compose

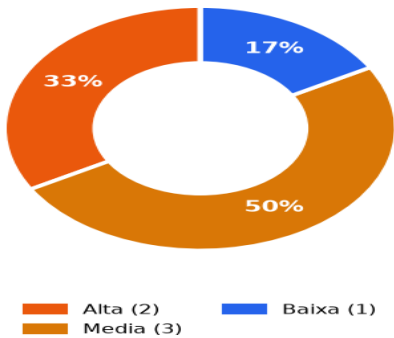
Metodologia: Revisão estática de código-fonte, análise de configuração, histórico git e mapeamento sistemático de rotas vs. gates de autorização.

1. Resumo Executivo

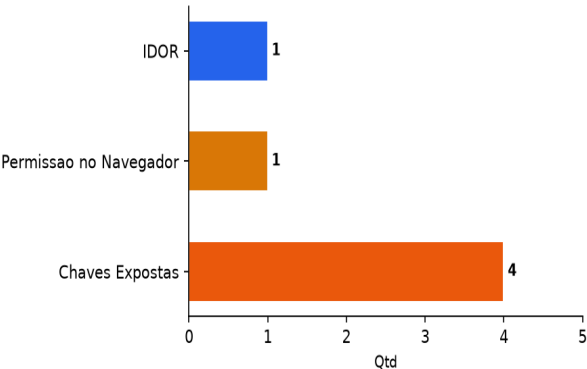
A auditoria identificou 6 achados em 4 categorias de risco. Nenhuma vulnerabilidade crítica foi encontrada. A camada de autenticação e autorização do backend está corretamente implementada na maioria dos casos. Os riscos principais concentram-se em configuração e exposição de credenciais.

Severidade	Qtd	IDs
Alta	2	F1, F2
Media	3	F3, F4, F5
Baixa	1	F6
Informativa	0	—

Achados por Severidade



Achados por Categoria



2. Pontos Fortes

OK Auth + RBAC no backend — jwtVerify() em todas as rotas protegidas

api/src/plugins/auth.ts L51-76: authorize() verifica token e role. Rotas: POST /events (ORGANIZER), POST /orders (CUSTOMER), GET /tickets/me (CUSTOMER), POST /tickets/validate (DOORMAN), GET /tmdb/search (ORGANIZER).

OK Isolamento de dono nos ingressos — GET /tickets/me

api/src/routes/tickets.ts L16: where: { ownerId: request.user.sub } — filtra apenas ingressos do usuário autenticado.

OK customerId e organizerId extraídos do JWT, nunca do body

orders.ts L29: customerId = request.user.sub | events.ts L60: organizerId: request.user.sub.

OK Anti-double-booking com SELECT FOR UPDATE

orders.ts L37: tx.\$queryRaw`SELECT id FROM Event WHERE id=... FOR UPDATE` — lock de linha para race condition.

OK Senha com bcrypt rounds=10 e erro genérico no login

users.ts L38 + sessions.ts L36: bcrypt.hash/compare. Mensagem 'E-mail ou senha inválidos' não revela qual campo errou.

OK QR Code assinado com JWT — forjamento impossível sem o segredo

tickets.ts L36: app.jwt.sign(ticketPayload). Validação verifica payload.type !== 'ticket'.

OK Cadastro sempre cria CUSTOMER — papéis privilegiados só via seed

users.ts L45: role: Role.CUSTOMER hardcoded, não lido do body.

OK Cookie httpOnly + SameSite por ambiente

sessions.ts L48-54: httpOnly:true, sameSite:'none'+secure:true em produção.

OK Helmet + CORS + validação Zod em todas as rotas

app.ts L33-35: @fastify/helmet, @fastify/cors. Todas as rotas usam safeParse antes de processar.

OK Zero innerHTML/dangerouslySetInnerHTML no frontend

Busca em todo o frontend: nenhuma ocorrência de XSS sink. Dados renderizados via JSX (auto-escape).

OK api/.env nunca commitado no histórico git

git log --all: apenas api/.env.example trackeado. .gitignore correto em cada subdiretório.

3. Achados Detalhados

Sev.	ID	Arquivo:Linha	Descrição
Alta	F1	api/.env L.6-7	TMDB_API_KEY real e JWT_SECRET fraco em arquivo .env local
Alta	F2	api/src/config/env.ts L.16	JWT_SECRET com default fraco no schema Zod — sem validação anti-default
Media	F3	web/lib/user-context.tsx L.26, 33, 40, 47	Senhas das contas seed hardcoded no bundle JS do frontend
Media	F4	api/src/config/env.ts L.11, 15	CORS_ORIGIN=* e DATABASE_URL com credenciais padrão nos defaults
Media	F5	web/app/page.tsx L.117-122	Link Publicar Evento visível para todos sem gate de papel no frontend
Baixa	F6	api/src/routes/tickets.ts L.77-122	DOORMAN pode validar ingressos de qualquer evento (sem verificação de posse)

F1 — TMDB_API_KEY real e JWT_SECRET fraco em arquivo .env local

Categoria: Chaves Expostas

Severidade: Alta

Arquivo: api/.env L.6-7

Descrição:

O arquivo api/.env contém a chave real da API TMDb (f7b09643311c852a39a9fdb49c47f6e) e o JWT_SECRET com valor padrão 'chave-provisoria-trocar-em-producao'. O risco primário é um commit acidental que exporia as credenciais.

Trecho:

```
TMDB_API_KEY=f7b09643311c852a39a9fdb49c47f6e
JWT_SECRET=chave-provisoria-trocar-em-producao
```

Explorabilidade:

Se comprometido, qualquer um pode usar a chave TMDb (abuso de cota) e assinar JWTs arbitrários impersonando qualquer usuário.

F2 — JWT_SECRET com default fraco no schema Zod — sem validação anti-default

Categoria: Chaves Expostas

Severidade: Alta

Arquivo: api/src/config/env.ts L.16

Descrição:

JWT_SECRET: z.string().min(10).default('chave-provisoria-trocar-em-producao'). Se a variável não for definida em produção, o app sobe silenciosamente com um segredo público.

Trecho:

```
JWT_SECRET: z.string().min(10).default('chave-provisoria-trocar-em-producao'),
```

Explorabilidade:

Atacante pode forjar JWTs válidos com qualquer sub/role usando o segredo público. Desabilita toda autenticação.

F3 — Senhas das contas seed hardcoded no bundle JS do frontend

Categoria: Chaves Expostas

Severidade: Media

Arquivo: web/lib/user-context.tsx L.26,33,40,47

Descrição:

PRESET_USERS contem password:'123456' para todas as contas (CUSTOMER x2, ORGANIZER, DOORMAN). Este código compila para o bundle JS servido ao navegador.

Trecho:

```
{ email: 'organizador@eventos.com', password: '123456', role: 'ORGANIZER' },
```

Explorabilidade:

Qualquer pessoa pode abrir DevTools, ler o bundle JS e obter login direto como ORGANIZER ou DOORMAN escalando privilégios.

F4 — CORS_ORIGIN=* e DATABASE_URL com credenciais padrão nos defaults

Categoria: Chaves Expostas

Severidade: Media

Arquivo: api/src/config/env.ts L.11,15

Descrição:

CORS_ORIGIN default '*' + credentials:true permite qualquer origem. DATABASE_URL default usa postgres:postgres.

Trecho:

```
CORS_ORIGIN: z.string().default('*'),  
DATABASE_URL: z.string().url().default('postgres://postgres:postgres@...')
```

Explorabilidade:

CORS aberto com credentials:true permite CSRF em qualquer domínio. Credenciais trivialmente adivinháveis.

F5 — Link Publicar Evento visível para todos sem gate de papel no frontend

Categoria: Permissão no

Navegador

Severidade: Media

Arquivo: web/app/page.tsx L.117-122

Descrição:

O botão '+ Publicar Evento' na home não verifica currentUser?.role === 'ORGANIZER'. O backend protege corretamente, mas a UI não.

Trecho:

+ Publicar Evento

Explorabilidade:

Impacto direto baixo (backend bloqueia). Expõe a existência da rota e confunde usuários com outros papéis.

F6 — DOORMAN pode validar ingressos de qualquer evento (sem verificacao de posse)

Categoria: IDOR

Severidade: Baixa

Arquivo: api/src/routes/tickets.ts L.77-122

Descrição:

POST /tickets/validate valida apenas role=DOORMAN, sem verificar se o porteiro esta atribuido ao eventId informado no corpo.

Trecho:

```
// Sem verificacao de posse do evento pelo DOORMAN
const ticket = await app.prisma.ticket.findUnique({ where: { code } })
```

Explorabilidade:

DOORMAN de evento A pode invalidar ingressos do evento B, causando negacao de acesso para portadores legitimos.

4. Recomendacoes Priorizadas

P1**Tornar JWT_SECRET obrigatorio sem default e validar na startup**

Qualquer default publico de segredo derruba toda a autenticacao.

P1**Revogar e rotacionar TMDB_API_KEY comprometida**

Chave real esta no arquivo .env local — risco se commitada.

P2**Remover senhas de PRESET_USERS do bundle JavaScript**

Credenciais de contas privilegiadas expostas no codigo-fonte publico.

P2**Eliminar CORS_ORIGIN=* e DATABASE_URL com defaults fracos**

CORS wildcard + credentials=true permite CSRF de qualquer origem.

P2**Adicionar gate de papel no link Publicar Evento da home**

Inconsistencia de UI: nao-ORGANIZER ve botao que o backend recusara.

P3**Vincular DOORMAN ao evento especifico na validacao**

DOORMAN pode invalidar ingressos de eventos alheios.

P3**Implementar secret scanning no CI (gitleaks/trufflehog)**

Detecta automaticamente commits acidentais de segredos.

5. Issues para o GitHub

Issues prontas para copiar e colar no repositório GitHub.

ISSUE 1: [Segurança] JWT_SECRET com default fraco e sem validação anti-default na startup

--- ISSUE 1 ---

Título: [Segurança] JWT_SECRET com default fraco e sem validação anti-default na startup

Labels: security, alta

Achados: F1, F2

Descrição

1. api/.env (L7): JWT_SECRET=chave-provisoria-trocar-em-producao
2. api/src/config/env.ts (L16): .default('chave-provisoria-trocar-em-producao') – app sobe sem erro com segredo público.

Impacto

Atacante pode forjar JWTs com qualquer sub/role, impersonando qualquer usuário incluindo ORGANIZER e DOORMAN.

Correção

1. Remover .default() do JWT_SECRET no env schema
2. Adicionar validação que rejeite valores da lista negra
3. Rotacionar o segredo em todos os ambientes
4. Gerar segredo forte: openssl rand -base64 64

Critérios de Aceite

- [] JWT_SECRET obrigatório sem fallback
- [] Startup falha se valor igual ao default
- [] Segredo rotacionado em staging e produção
- [] Teste verifica que API recusa o default

--- FIM ISSUE 1 ---

ISSUE 2: [Segurança] TMDB_API_KEY e credenciais padrao expostas em configs

--- ISSUE 2 ---

Título: [Segurança] TMDB_API_KEY e credenciais padrao expostas em configs

Labels: security, alta

Achados: F1, F4

Descrição

api/.env L6: TMDB_API_KEY real em arquivo commitavel.

env.ts L11,15: CORS_ORIGIN default '*' + DATABASE_URL com postgres:postgres.

Impacto

Chave TMdb exposta: abuso de cota. CORS wildcard + credentials: CSRF. Credenciais DB adivinhaveis.

Correcao

1. Revogar e rotacionar TMDB_API_KEY
2. Remover defaults de DATABASE_URL e CORS_ORIGIN
3. Configurar CORS_ORIGIN explicitamente por ambiente
4. Adicionar secret scanning no CI

Critérios de Aceite

- [] TMDB_API_KEY rotacionada
- [] Sem defaults de segredos no env schema
- [] CORS_ORIGIN configurado com lista de origens
- [] CI com truffleshog/gitleaks

--- FIM ISSUE 2 ---

ISSUE 3: [Segurança] Senhas de contas seed no bundle JavaScript do frontend

--- ISSUE 3 ---

Título: [Segurança] Senhas de contas seed no bundle JavaScript do frontend

Labels: security, media

Achados: F3

Descrição

web/lib/user-context.tsx L26,33,40,47: PRESET_USERS com password:'123456' para todas as contas incluindo ORGANIZER e DOORMAN. Compilado no bundle publico.

Impacto

Qualquer visitante obtém credenciais de contas privilegiadas via DevTools.

Correcao

1. Remover senhas do array PRESET_USERS
2. Feature de login rapido so em NODE_ENV=development
3. Alterar senhas das contas seed em todos os ambientes

Critérios de Aceite

- [] Senhas removidas do codigo-fonte frontend
- [] Contas seed com senhas fortes
- [] Login por preset desabilitado em producao
- [] Bundle não contém strings de senha

--- FIM ISSUE 3 ---

ISSUE 4: [Segurança] DOORMAN pode validar ingressos de qualquer evento (IDOR de ação)

--- ISSUE 4 ---

Título: [Segurança] DOORMAN pode validar ingressos de qualquer evento (IDOR de ação)

Labels: security, baixa

Achados: F6

Descrição

api/src/routes/tickets.ts L77-122: POST /tickets/validate valida apenas role=DOORMAN. Sem verificação de que o porteiro está atribuído ao eventId informado.

Impacto

DOORMAN de evento A pode marcar como USED ingressos do evento B, negando acesso a portadores legítimos.

Correção

1. Criar campo assignedEventId no modelo User
2. Verificar eventId === request.user.assignedEventId na rota
3. Alternativa: incluir eventId no JWT do DOORMAN no login

Critérios de Aceite

- [] DOORMAN só valida ingressos de seu evento
- [] Teste: DOORMAN de evento A recebe 403 para evento B
- [] Documentação do fluxo de atribuição

--- FIM ISSUE 4 ---

ISSUE 5: [Segurança] Botão Publicar Evento sem gate de papel no frontend (home)

--- ISSUE 5 ---

Título: [Segurança] Botão Publicar Evento sem gate de papel no frontend (home)

Labels: security, média

Achados: F5

Descrição

web/app/page.tsx L117-122: link '+ Publicar Evento' visível para todos sem verificar role === ORGANIZER.

Impacto

Baixo impacto direto. Expõe rota e confunde usuários com outros papéis.

Correção

Envolver o Link com {currentUser?.role === 'ORGANIZER' && ...}. Seguir padrão já usado na Navbar.tsx L82-95.

Critérios de Aceite

- [] Botão oculto para não-ORGANIZER
- [] Teste: visitante e CUSTOMER não veem o botão
- [] ORGANIZER continua vendo normalmente

--- FIM ISSUE 5 ---